COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	8/4/26	**DATE SUBMITTED:**	8/4/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

Continuing from the seatwork ILOA and B1..

B.2. Stacks using Linked Lists

```
1   #include <iostream>
2
3   class Node {
4       public:
5           int data;
6           Node* next;
7
8   };
9
10  Node *head = NULL, *tail = NULL;
```

Node Class

This class is the basic class template for all linked lists.

```
12  void push(int newData) {
13      Node *newNode = new Node;
14      newNode->data = newData;
15      newNode->next = head;
16
17      if(head == NULL){
18          head = tail = newNode;
19      } else {
20          newNode->next = head;
21          head = newNode;
22      }
23  }
```

Node Creation

This creates a new node and assigns it as the new 'head' which in this case is the last-in element.

```
25    int pop() {
26        int tempVal;
27        Node* temp;
28
29        if(head == NULL) {
30            head = tail = NULL;
31            std::cout << "Stack underflow!\n";
32            return -1;
33        } else {
34            temp = head;
35            tempVal = head->data;
36            head = head->next;
37            delete temp;
38            return tempVal;
39        }
40    }
```

Pop() Function

This function only deletes the head data and returns the deleted value as it is executed.

```
void Top() {
    if(head == NULL) {
        std::cout << "Stack is empty\n";
    } else {
        std::cout << head->data << std::endl;
    }
}
```

Top() Function

This function prints—instead of returning—the data in the head node.

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.

```
void displayStack() {
    if(head == NULL) {
        std::cout << "Stack underflow!\n";
    } else {
        Node* temp = head;
        while(temp != NULL) {
            std::cout << temp->data << std::endl;
            temp = temp->next;
        }
    }
}
```

This function is a basic while-loop that prints the data of the head up to the tail until we get a null value.

- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

```
After the first PUSH top of stack is: 1
After the second PUSH top of stack is: 5
After the first POP operation, top of stack is: 1
After the second POP operation, top of stack is: Stack is empty
Stack underflow!

Pushing 1 to 5 into the stack:
1
2
3
4
5

Testing print stack:
5
4
3
2
1
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:

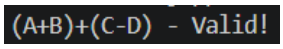
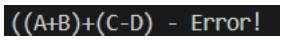
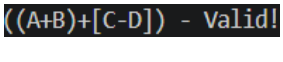
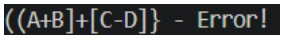
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {
    a) If the character read is not a symbol to be balanced, ignore it.
    b) If the character is an opening symbol, push it onto the stack.
    c) If it is a closing symbol:
        i) Report an error if the stack is empty.
        ii) Otherwise, pop the stack.
    d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

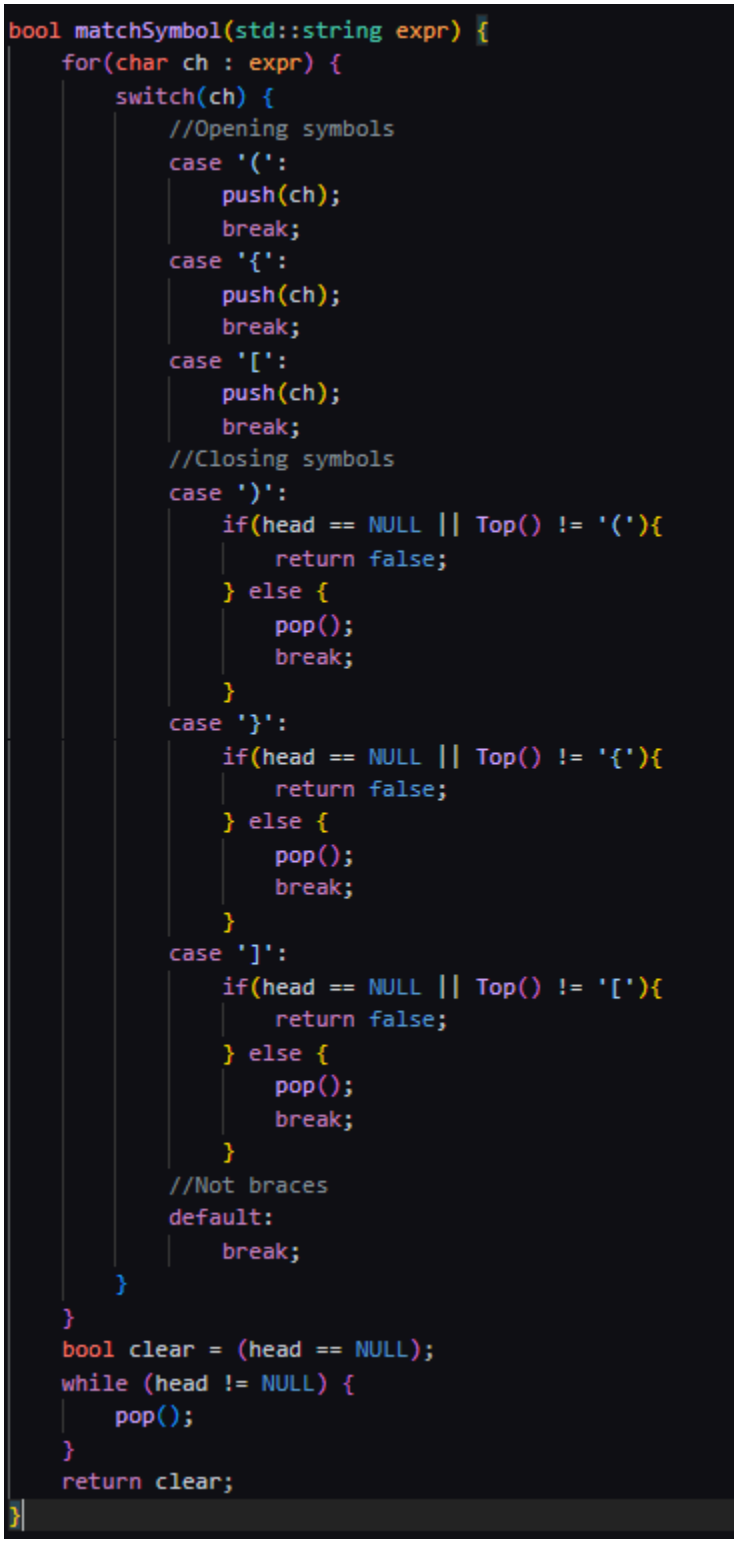
Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		This is correct since all the braces were properly closed.
((A+B)+(C-D)	N		This is correct because there is a missing closing parenthesis.
((A+B)+[C-D])	Y		This is correct even if one of the braces was different since it was still properly closed.
((A+B]+[C-D]}	N		The opening parenthesis cannot match with a closing curly brace

```cpp
bool matchSymbol(std::string expr) {
    for(char ch : expr) {
        switch(ch) {
            //Opening symbols
            case '(':
                push(ch);
                break;
            case '{':
                push(ch);
                break;
            case '[':
                push(ch);
                break;
            //Closing symbols
            case ')':
                if(isEmpty() || Top() != '('){
                    return false;
                } else {
                    pop();
                    break;
                }
            case '}':
                if(isEmpty() || Top() != '{'){
                    return false;
                } else {
                    pop();
                    break;
                }
            case ']':
                if(isEmpty() || Top() != '['){
                    return false;
                } else {
                    pop();
                    break;
                }
            //Not braces
            default:
                break;
        }
    }
    //return after whole string is read
    if(isEmpty()) return true;
    return false;
}
```

Array Stack Function



```
bool matchSymbol(std::string expr) {
    for(char ch : expr) {
        switch(ch) {
            //Opening symbols
            case '(':
                push(ch);
                break;
            case '{':
                push(ch);
                break;
            case '[':
                push(ch);
                break;
            //Closing symbols
            case ')':
                if(head == NULL || Top() != '('){
                    return false;
                } else {
                    pop();
                    break;
                }
            case '}':
                if(head == NULL || Top() != '{'){
                    return false;
                } else {
                    pop();
                    break;
                }
            case ']':
                if(head == NULL || Top() != '['){
                    return false;
                } else {
                    pop();
                    break;
                }
            //Not braces
            default:
                break;
        }
    }
    bool clear = (head == NULL);
    while (head != NULL) {
        pop();
    }
    return clear;
}
```

Linked List Stack Function
(Modified from array)

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?
  - They work differently in the way we access the stack contents and the memory allocation. With an array, there is fixed memory as the program runs but with a linked list there's none. We also check the emptiness differently, with arrays we need to set an index to -1 but with a linked list, we just look if the head is equal to null.

Provide the following:

- Summary of lessons learned
  - I have learned how to make stacks using arrays and linked list using custom functions.
- Analysis of the procedure
  - The procedure taught me how to create the functions and showed me how they work along with the logic behind them.
- Analysis of the supplementary activity
  - The supplementary activity is a good exercise to master the simple applications of a stack with linked lists and arrays.
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for Improvement?
  - I think I did pretty well and that my code is mostly correct with as few errors as I can have. Since I got the expected result, I will take that as a completed task. I think I can improve more on the syntax and shorthand methods in c++ since I forget them often.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Gemini

Purpose of Use:

Error searching

Extent of Use:

The AI tool was used to pinpoint exactly where the error was and what was causing it, enabling me to fix the bugs.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.